Detailed Explanation of Sections 3.3, 3.3.1, and 3.3.2

Here is an elaborate explanation of Sections 3.3, 3.3.1, and 3.3.2 along with three coding examples for each section to illustrate the concepts.

1 Section 3.3: Matrix Multiplication

Matrix multiplication is a fundamental operation where two matrices are multiplied by taking the rows of the first matrix and multiplying them element-by-element with the columns of the second matrix, summing the products. This process is known as Falk's scheme.

In Python's NumPy, you can perform matrix multiplication using the infix operator `@`, the `np.matmul(A,B)` function, or `np.multi_dot([A,B,C,...])` for multiple matrices. A critical mathematical rule to remember is that matrix multiplication is **not commutative**. This means that the order of multiplication drastically changes the outcome (i.e., $A \cdot B \neq B \cdot A$). In electrical engineering, this is heavily used to calculate input voltages and currents (U_1, I_1) of cascaded two-port networks (like a π-substitute circuit) using "A parameters" (catenary shape).

1.1 Examples for Section 3.3

Example 1: Demonstrating Non-Commutativity
This example defines two simple 2x2 matrices and multiplies them in both directions to prove that the outputs are different.

```
import numpy as np

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

# Multiplying A * B
C = A @ B
# Multiplying B * A
D = B @ A

print("A @ B =\n", C) # Results in [[19, 22], [43, 50]]
print("B @ A =\n", D) # Results in [[23, 34], [31, 46]]
```

Example 2: Calculating Input Variables (A Parameters)
This example uses the catenary shape (A parameters) to find the required input voltage (U_1) and current (I_1) for a π-substitute circuit given known output variables.

```
import numpy as np

R1 = 1; R2 = 2; R3 = 1
U2 = 1; I2 = 1

# Defining the partial matrices for the resistors
A1q = np.array([[1, 0], [1/R1, 1]])
Al  = np.array([[1, R2], [0, 1]])
A2q = np.array([[1, 0], [1/R3, 1]])

# Calculating the total system matrix A
A = A1q @ Al @ A2q

# Output vector
b = np.array([[U2], [I2]])

# Calculating input vector E
E = A @ b
print(f"Input Voltage U1: {E[0,0]:.2f} V") # Output: 5.00 V
print(f"Input Current I1: {E[1,0]:.2f} A") # Output: 7.00 A
```

Example 3: Cascading Matrices with `multi_dot`
When calculating large networks, chaining multiple `@` operators can become hard to read. NumPy provides `multi_dot` to chain them together cleanly, preserving the left-to-right calculation order.

```
import numpy as np

# Three random transformation matrices
M1 = np.array([[1, 0], [0, 2]])
M2 = np.array([[0, 1], [1, 0]])
M3 = np.array([[2, 0], [0, 2]])

# Using multi_dot for chained matrix multiplication
result = np.linalg.multi_dot([M1, M2, M3])

print("Chained Result:\n", result)
```

2 Section 3.3.1: Chain Shape with B Parameters

While A parameters calculate input variables based on known output variables, **B parameters** do the exact opposite: they calculate the output variables (U_2, I_2) from known input variables (U_1, I_1).

Mathematically, the B parameter matrix can be determined from the inverse of the A matrix. In practice, setting up the B parameters for a π-substitute circuit looks almost identical to setting up the A parameters, except the parameters located in the secondary diagonal of the matrices are given a negative sign.

2.1 Examples for Section 3.3.1

Example 1: Calculating Output Variables (B Parameters)
This directly applies the B parameters to calculate the output of a π-substitute circuit where the input voltage is 5V and the input current is 7A.

```
import numpy as np

R1 = 1; R2 = 2; R3 = 1
U1 = 5; I1 = 7

# Defining B parameter matrices (note the negative signs on the secondary
    diagonals)
B1q = np.array([[1, 0], [-1/R1, 1]])
B2l = np.array([[1, -R2], [0, 1]])
B3q = np.array([[1, 0], [-1/R3, 1]])

# Total matrix B
B = B1q @ B2l @ B3q

# Input vector
b = np.array([[U1], [I1]])

# Calculating output vector E
E = B @ b
print(f"Output Voltage U2: {E[0,0]:.2f} V") # Output: 1.00 V
print(f"Output Current I2: {E[1,0]:.2f} A") # Output: 1.00 A
```

Example 2: Analyzing a Single Series Resistor
If we simplify the network to just one series resistor (R_2) and remove the transverse resistors, the B-parameter calculation becomes much smaller but follows the same rules.

```
import numpy as np

R2 = 5 # 5 Ohm series resistor
U1 = 12 # 12V input
I1 = 2 # 2A input

# B matrix for just a series resistor
B2l = np.array([[1, -R2], [0, 1]])
input_vector = np.array([[U1], [I1]])

# Output calculation
output_vector = B2l @ input_vector
print(f"Output Voltage: {output_vector[0,0]} V") # Drops by I*R (12 - 2*5 = 2V)
print(f"Output Current: {output_vector[1,0]} A") # Remains 2A
```

Example 3: Verifying the Inverse Relationship
The text notes that B parameters are derived from the inverse matrix of A. We can use NumPy's `linalg.inv()` function to explicitly invert the A matrix from Section 3.3 to find the output variables, proving the mathematical concept.

```
import numpy as np

# Recreating matrix A from the earlier pi-circuit example
A = np.array([[3, 2], [4, 3]])
input_vector = np.array([[5], [7]])

# Finding B by inverting A
B_inverted = np.linalg.inv(A)

# Calculating output using the inverted matrix
output = B_inverted @ input_vector
print("Output via Inverse Matrix:\n", output) # Yields [[1], [1]]
```

3 Section 3.3.2: Calculating the Energy of a Rotating Rigid Body in Space

Matrix multiplication is also crucial in mechanical physics, such as calculating the rotational energy of a rigid body in three-dimensional space.

The formula for rotational energy requires multiplying three components together:

$$E_{rot} = 0.5 \cdot \omega^T \cdot I \cdot \omega$$

Here, ω is the column vector of angular velocity, ω^T is the transposed (row) vector of the angular velocity, and I is the 3x3 inertia tensor matrix (which incorporates the body's mass and its location coordinates). Because of non-commutativity, the sequence of "row vector multiplied by 3x3 matrix multiplied by column vector" is strictly mandatory.

3.1 Examples for Section 3.3.2

Example 1: Rotational Energy Around the Z-Axis
This example mirrors the document's scenario where a 6kg mass rotates exclusively around the z-axis, requiring a complex 3x3 inertia tensor to interact with the vectors.

```
import numpy as np

m = 6 # mass in kg
x = 1; y = 0; z = 0 # distance coordinates
wx = 0; wy = 0; wz = 1 # angular velocity components

# Row vector (w_Z), Inertia Tensor (I), Column vector (w_S)
w_Z = np.array([wx, wy, wz])
I = m * np.array([
    [y**2 + z**2, -x*y, -x*z],
    [-x*y, x**2 + z**2, -y*z],
    [-x*z, -y*z, x**2 + y**2]
])
w_S = np.array([[wx], [wy], [wz]])

# Erot = 0.5 * (row vector @ tensor @ column vector)
Erot = 0.5 * w_Z @ I @ w_S
print(f"Rotational Energy: {Erot[0]:.2f} joules") # Output: 3.00 joules
```

Example 2: Using the `.T` (Transpose) Method

Instead of manually defining a row vector and a separate column vector for angular velocity, it is much more efficient in Python to define just the column vector and use NumPy's `.T` attribute to transpose it directly within the equation.

```
import numpy as np

m = 6; x = 1; y = 0; z = 0
wx = 0; wy = 0; wz = 1

# Define ONLY the column vector
w_S = np.array([[wx], [wy], [wz]])

I = m * np.array([
    [y**2 + z**2, -x*y, -x*z],
    [-x*y, x**2 + z**2, -y*z],
    [-x*z, -y*z, x**2 + y**2]
])

# Use the .T transpose property directly in the calculation
Erot = 0.5 * w_S.T @ I @ w_S
print(f"Rotational Energy via Transpose: {Erot[0,0]:.2f} joules")
```

Example 3: Complex Multi-Axis Rotation

We can adjust the angular velocity vector to simulate the object tumbling across multiple axes simultaneously (e.g., spinning around both the X and Z axes).

```
import numpy as np

m = 6; x = 1; y = 2; z = 0 # Moved the mass further out in the x-y plane
wx = 2; wy = 0; wz = 1 # Rotating on two axes simultaneously

w_S = np.array([[wx], [wy], [wz]])

# Inertia tensor dynamically reacts to the new position
I = m * np.array([
    [y**2 + z**2, -x*y, -x*z],
    [-x*y, x**2 + z**2, -y*z],
    [-x*z, -y*z, x**2 + y**2]
])

# The same core formula effortlessly handles the multi-axis math
Erot = 0.5 * w_S.T @ I @ w_S
print(f"Complex Rotational Energy: {Erot[0,0]:.2f} joules")
```

System of Linear Equations (Sections 3.4)

1 Section 3.4: Linear Systems of Equations

Linear systems of equations are essential tools in engineering used to calculate unknowns in interconnected systems. For example, they are used in electrical engineering to find mesh currents and node voltages, and in structural analysis to determine member forces in trusses.

Solving systems with n unknowns requires significant computational power if done manually. The NumPy module provides a highly optimized function, `solve()`, which handles these calculations effortlessly for both real and complex coefficients. This tool functions similarly to commercial engineering software like MATLAB.

1.1 Examples for Section 3.4: Core Concepts

Example 1: The Basic Syntax of `solve()`
This example demonstrates the fundamental required imports and the basic structure of passing a matrix and a vector to the `numpy.linalg.solve()` function.

```
import numpy as np
from numpy.linalg import solve

# A generic 2x2 matrix A and vector b
A = np.array([[2, 1], [1, 3]])
b = np.array([5, 5])

# The solve function calculates the intersection/solution
unknowns = solve(A, b)
print("Solution:", unknowns) # Output: [2. 1.]
```

Example 2: Mathematical Definition via Matrix Inversion
Mathematically, a linear system $A \cdot x = b$ can also be solved by finding the inverse of matrix A and multiplying it by vector b ($x = A^{-1} \cdot b$). While `solve()` is generally preferred for performance, this example demonstrates the mathematical foundation.

```
import numpy as np

A = np.array([[4, 3], [3, 2]])
b = np.array([10, 7])

# Step 1: Find the inverse of A
A_inv = np.linalg.inv(A)

# Step 2: Multiply the inverse by b
x = A_inv @ b
print("Solution using Inverse:", x) # Output: [1. 2.]
```

Example 3: Verifying the Solution
After finding the unknowns, it is good engineering practice to verify the results by plugging them back into the original equation ($A \cdot x$). The result should perfectly match the inhomogeneity vector b.

```
import numpy as np

A = np.array([[5, -2], [-3, 4]])
```

```
b = np.array([12, 4])
x = np.linalg.solve(A, b)

# Verify by multiplying A and the solved x
verification = A @ x
print("Original b:", b)
print("A @ x:     ", verification)
# Both output [12. 4.], confirming the solution is correct.
```

2 Section 3.4.1: Systems of Equations with Real Coefficients

In a system of linear equations with real coefficients, the equations are represented as a matrix product. The components are:

- **Coefficient Matrix (A):** Contains the parameters a_{11} to a_{mn}.
- **Solution Vector (x):** The column vector containing the unknown variables we are trying to find.
- **Inhomogeneity Vector (b):** The column vector on the right-hand side of the equal sign.

One critical behavior of NumPy's `solve(A, b)` function is how it handles data types. Even if the coefficient matrix and the inhomogeneity vector consist entirely of integers, NumPy automatically converts the elements of the resulting solution vector into floats (specifically `numpy.float64`). This ensures that fractional solutions are not truncated. Furthermore, if even one integer in the input array is declared as a float (e.g., `2.` instead of `2`), NumPy automatically converts all other elements in that array to floats.

2.1 Examples for Section 3.4.1

Example 1: Solving a 3x3 Real System
This example replicates the book's demonstration of solving a 3-variable system (Listing 3.16).

```
import numpy as np

# Coefficient matrix A (3x3)
A = np.array([[1,  1, 1],
              [2, -2, 3],
              [3, -4, 2]])

# Inhomogeneity vector b (3x1)
b = np.array([6, 7, 1])

# Calculate solution
solution = np.linalg.solve(A, b)
print("Solution x:", solution) # Output: [1. 2. 3.]
```

Example 2: Demonstrating Automatic Data Type Conversion
This example highlights how NumPy handles types, proving that integer inputs yield float outputs.

```
import numpy as np

A = np.array([[2, 1], [1, -1]]) # Integers
b = np.array([8, 1])            # Integers

x = np.linalg.solve(A, b)

print("Type of A element:", type(A[0,0]))       # Output: <class 'numpy.int64'>
print("Type of b element:", type(b[0]))         # Output: <class 'numpy.int64'>
print("Type of solution element:", type(x[0]))  # Output: <class 'numpy.float64
    '>
```

Example 3: Structural Member Forces (Real Scenario)
Imagine a simple structural node where two truss members meet. The equilibrium equations in the X and Y directions form a 2x2 linear system with real geometric coefficients.

```
import numpy as np

# Force equilibrium matrix based on member angles (cos and sin components)
# F1*cos(45) - F2*cos(30) = 0
# F1*sin(45) + F2*sin(30) = 1000 (external load)
A = np.array([[np.cos(np.pi/4), -np.cos(np.pi/6)],
              [np.sin(np.pi/4),  np.sin(np.pi/6)]])
b = np.array([0, 1000])

forces = np.linalg.solve(A, b)
print(f"Force in Member 1: {forces[0]:.2f} N")
print(f"Force in Member 2: {forces[1]:.2f} N")
```

3 Section 3.4.2: Systems of Equations with Complex Coefficients

In alternating current (AC) electrical networks, resistances are replaced by complex impedances because components like inductors and capacitors introduce phase shifts. The system of equations handles these complex values natively.

When formulating the matrix for mesh analysis in an AC network:

- The **main diagonal** contains the sum of all impedances within a specific mesh.
- The **secondary diagonals** contain the common impedances shared between two meshes.
- If two meshes do not share an impedance, a `0` is entered.
- Shared impedances on the secondary diagonals are given a negative sign and are mirrored across the main diagonal, creating a symmetric matrix.

NumPy's `solve()` function processes complex variables natively without requiring any special configuration. Python represents the imaginary unit with a `j` (e.g., `1+2j`).

3.1 Examples for Section 3.4.2

Example 1: A 2-Mesh Complex AC Circuit
This example models a simple AC circuit with two loops using complex impedances.

```
import numpy as np

# Define complex voltages
U1 = 120 + 0j
U2 = 0 + 0j

# Define complex impedances (R + jX)
Z1 = 10 + 5j
Z2 = 2 - 4j
Z3 = 5 + 5j

# Matrix setup based on mesh rules:
# Main diagonal = sum of loop impedances.
# Secondary = shared impedances (negative).
Z_matrix = np.array([[Z1 + Z2, -Z2],
                     [-Z2, Z2 + Z3]])
U_vector = np.array([U1, U2])

# Solve for complex mesh currents
currents = np.linalg.solve(Z_matrix, U_vector)
print("Mesh 1 Current:", currents[0])
print("Mesh 2 Current:", currents[1])
```

Example 2: Using `enumerate` to Extract Real and Imaginary Parts
When dealing with arrays of complex solutions, it is often useful to separate the real and imaginary parts for readable output, exactly as demonstrated in Listing 3.17 of the text using `enumerate()`.

```
import numpy as np

Z = np.array([[2+3j, -1-1j],
              [-1-1j, 4+2j]])
U = np.array([10, -5])

currents = np.linalg.solve(Z, U)

# Iterating through the complex array
for k, I in enumerate(currents, start=1):
    # I.real and I.imag extract the respective floating-point components
    print("I%d = (%0.2f, %0.2fj) A" % (k, I.real, I.imag))
```

Example 3: Calculating Power from Complex Solutions
In AC networks, once you have solved for the complex currents, you can calculate the apparent power ($S = V \cdot I^*$) by using the complex conjugate of the calculated current.

```
import numpy as np

# Simple 1-equation complex system (Ohm's Law: V = I*Z)
Z = np.array([[4 + 3j]]) # 4 Ohms resistance, 3 Ohms reactance
U = np.array([230 + 0j]) # 230V source

I = np.linalg.solve(Z, U)

# Get the complex conjugate of the current using np.conj()
I_conjugate = np.conj(I[0])

# Calculate Apparent Power (S)
S = U[0] * I_conjugate
print(f"Apparent Power: {S.real:.2f} W (Active), {S.imag:.2f} VAR (Reactive)")
```